

磁石球と砂鉄の実装計画 (3D_CPP_magnet)

Context

MAGNET_SPEC.md をもとに、自作剛体エンジン (src/phys) へ磁石の球を追加する。仕様書から、会話で次のように変更した。

- 渦電流は入れない (§3.4, §4.3, T7, シーン 7 の旧案は削除)
- 場の方式: 磁場は格子に保存しない。源 (各磁性体の位置とモーメント) のリストを持ち、 B と ∇B を関数として毎回評価する
- 誘導磁化を統一式で入れ、砂鉄も磁石と同じ規則で動く物体にする (描画だけの砂鉄表示はやめる)
- このフォルダの src を使い、既存コードの変更は最小にする

前提の確認結果:

- src/ は ../3D_CPP/src と完全に同一 (diff -rq で差分なし)。test/phys は無いので MO (統一) は不要
- Physics Range (game.cpp) とベンチはこのフォルダに無いので対象外
- CMakeLists.txt とテストはこのフォルダに無い。../3D_CPP は CMake + Visual Studio 構成で、raylib は FetchContent で取得している

方針: 既存コードに触らない工夫

- World のインターフェースは変えない
 - サブステップは外側で world.step(dt/n) を n 回呼ぶ。中身は仕様書の substep と同じ
 - 磁力は各 step の直前に body->force と body->torque へ加算する。既存の流れでは integrateForces (world.cpp:88) が力を消費し、integratePositions (world.cpp:315) が 0 に戻す。これに乗るだけ
 - 静的剛体への力は integrateForces が捨てる (world.cpp:91-96) ので、分岐は不要
- 既存ファイルで変更するのは main.cpp だけ。物理コア (world.*, body.h, math3d.h, collision.*) は一切変更しない
- 磁気の物理はすべて新規の magnet.h/.cpp に閉じ込める
- 空間ハッシュや sweep などの高速化は今回入れない。broadphase も磁気も総当たりのままにする (砂鉄の数はそれで決まる。後述)

物理モデル (統一式)

すべての磁性体 i について、同じ式を使う ($K = \mu_0/4\pi = 1$)。

- モーメント: $\mathbf{m}_i = \mathbf{R}_i (\mathbf{p}_i + \chi_i \odot (\mathbf{R}_i^T \mathbf{B}_i))$ 、ただし $|\mathbf{m}_i| \leq m_{\text{sat}}$ (0 なら上限なし)
 - $\mathbf{p}$ は永久モーメント、 χ は磁化率 (ローカル主軸の対角 Vec3)
- 場: $\mathbf{B}_i = \sum_j \text{dipoleField}(\mathbf{x}_i - \mathbf{x}_j, \mathbf{m}_j)$
- 力とトルク: ペア (i, j) ごとに場の関数で計算する
 - $\mathbf{F}_j = G_{\{i \rightarrow j\}} \mathbf{m}_j$ 、 $\mathbf{F}_i = -\mathbf{F}_j$ 。作用・反作用で入れるので、運動量は厳密に保存する
 - $\boldsymbol{\tau}_j = \mathbf{m}_j \times \mathbf{B}_{\{i \rightarrow j\}}$ 、 $\boldsymbol{\tau}_i = \mathbf{m}_i \times \mathbf{B}_{\{j \rightarrow i\}}$
- 双極子の場の勾配 (対称でトレース 0) : $G = (3/r^4)[\mathbf{m} \hat{\mathbf{n}}^T + \hat{\mathbf{n}} \mathbf{m}^T + (\mathbf{m} \cdot \hat{\mathbf{n}})\mathbf{I} - 5(\mathbf{m} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}} \hat{\mathbf{n}}^T]$
- 近距離クランプ: $r_{\text{eff}} = \max(r, 0.9(R_i + R_j))$ (§3.3 と同じ)

- 全ペアの総当たりで計算する（カットオフなし）
- 誘導モーメント: 前サブステップの m を初期値にして Jacobi 反復する（既定はサブステップごとに 1 回）
- 磁気エネルギー: $\mathbf{U} = -\frac{1}{2} \sum \mathbf{p}_i \cdot \mathbf{B}_i$ 。線形で自己無撞着な状態なら厳密で、永久磁石だけなら通常の式に一致する

物体	$\mathbf{p}$ (ローカル)	χ (ローカル)	飽和 m_{sat}
磁石 球	$m_0 \hat{y}$ (m_0 は Γ から逆算)	0	0
砂鉄 粒	0	$\beta r^3 (1,1,1)$, $\beta \approx 0.7$	$0.5 \times (\text{磁石の単位体積あたりモーメント}) \times \text{粒の体積}$

- 砂鉄（磁鉄鉱）は丸い粒なので、**等方的な球**として扱う
 - トルクは 0 なので、回転の硬さの問題は起きない
 - 磁力線に沿ったトゲ（鎖）は、粒どうしの誘導で自然に出る
 - 針状にしたければ $\chi = (0, \alpha, 0)$ にするだけで、コードの変更は不要
- 単位とパラメータ (§3.2 のまま)
 - 磁石: 半径 0.25、 $\Gamma = 779$ 、 $\lambda = 100$ （シミュレーションの 1 秒 = 実時間 0.1 秒）、質量 0.5、 $m_0 = \sqrt{(\Gamma M g d^4 / 6)}$
 - 砂鉄粒: 半径 0.03、質量は磁石と同じ密度から決める
 - 相似則: β と m_{sat} はどちらも長さの比だけで決まるので、実物と相似になる

変更・追加するファイル

新規

1. **CMakeLists.txt**: `../3D_CPP/CMakeLists.txt` を元に作る
 - `physics` に `src/phys/magnet.cpp` を追加する
 - ターゲットは `demo`, `core_test`, `magnet_test`。 `test_app` は削除する
2. **tests/core_test.cpp**: `../3D_CPP/tests/core_test.cpp` をコピーする（既存エンジンの回帰 13 項目）
3. **src/phys/magnet.h / magnet.cpp**
 - 純粋関数:
 - `dipoleField(r, m), dipoleGradient(r, m) → Mat3, dipoleEnergy(r, m1, m2)`
 - `dipoleForce(r, m1, m2)`（検算用。§3.1 の閉じた式）
 - `momentFromGamma(Γ, M, d, g), gammaFromMaterial(Br, ρ, R, g)`
 - `struct MagneticBody { RigidBody* body; Vec3 permanentLocal, chiLocal; float saturation; Vec3 m, B; }`
 - `class MagnetSystem`
 - パラメータ: `minDistScale = 0.9, inducedIterations = 1`
 - `add(body, p, chi, sat)` は 1 本だけ用意する（磁石も砂鉄も同じ関数で登録）。`clear()`
 - `updateMoments(iters)`: 統一式で m を更新する
 - `applyForces()`: `updateMoments` → 全ペアで力とトルクを加算
 - `potentialEnergy()`: 現在位置で場を評価し直して計算する
 - `fieldAt(p)`: 任意の点の場（テスト・HUD 用）

- `structures(const World&):` 磁性体どうしの接触マニフォールドをグラフにし、連結成分ごとに鎖・輪・単体・その他を判定する (§6.4)
 - `bodies()`
 - `void stepMagnetic(World&, MagnetSystem&, float dt, int substeps):` for n: `ms.applyForces(); w.step(dt/n);`
 - `EnergyReport computeEnergy(const World&, const MagnetSystem&):` 並進・回転・重力・磁気
 - `outer(a, b)` は `magnet.cpp` 内の static 関数にする (`math3d.h` は触らない)
4. `tests/magnet_test.cpp: core_test.cpp` と同じ `check(name, ok, detail)` 形式。raylib は不要。
T3 の結果は CSV でも出す

既存の変更（このファイルだけ）

5. `main.cpp`

- Demo に追加するメンバ: `MagnetSystem magnets; int substeps = 1; std::vector<char> magnetic; (id を添字にする), Model hemiModel`
- `buildScene:` 先頭で `magnets.clear()`、`substeps = 1`、重力を元に戻す。case 4, 5, 6 を追加する
- `stepPhysics(dt)` を追加する。既存の `demo.world.step(FIXED_DT)` 2 か所 (`main.cpp:292`, `main.cpp:296`) をこれに置き換える
 - 磁性体が 0 個で `substeps = 1` なら、今までと同じ動作になる
- `draw()`
 - 磁性体は通常の描画ループで飛ばす
 - 磁石は `GenMeshHemiSphere` の半球 2 つで描く (N 極が赤、S 極が青)。p は常にローカル +Y なので、体の姿勢と「X 軸まわり 180°」の 2 通りで描ける
 - 砂鉄粒は小さな暗い球で描き、`|m|` に応じて明るさを変える
- `shoot()` (`main.cpp:178`): シーン 5~7 では、弾を磁石球にする (`magnets.add` を 1 行追加し、向きはランダム)。シーン 1~4 は今までどおり
- キー操作
 - 1~7: シーン切り替え
 - Z: 重力 0 の切り替え
 - T: 砂鉄を叩く (小さなランダムインパルス)
 - M: シーン 7 の配置切り替え
- HUD: エネルギーの内訳と合計、鎖・輪・単体の数、`Γ`、`substeps`、「実時間 × 0.1」

デモシーン

- 5: ばらまき
 - 磁石球 64 個を、ランダムな位置と向きで床の上空から落とす
 - `substeps 8`。Z で重力 0
- 6: 鎖か輪か
 - `N = 3~8` の円弧 (隙間を 1 か所あける) を横一列に並べ、同時に放す
 - 各群の上に「鎖 / 輪」と、群内のエネルギーの現在値と理論値 (付録 A.1) を表示する。群内エネルギーは `main.cpp` 側で `dipoleEnergy` を足して求める。表示位置は `GetWorldToScreen` で決める
- 7: 砂鉄
 - 床に静的な磁石球 (モーメントは水平) を置き、周囲に砂鉄粒をまく

- **M** で配置を切り替える: 1 個 / 2 個 (N-S) / 2 個 (N-N)
- **T** で叩いて摩擦から解放する (実験で紙を叩くのと同じ操作)
- substeps 8
- **粒の数**: 総当たりなので、コストは (粒数)² に比例する。見積もりでは 200~300 個程度が実時間の上限。まず 200 個で始め、HUD の physics ms を見て決める。実時間に間に合わない場合でも、既存の accumulator の上限 ([main.cpp:290](#)) により、スローになるだけで破綻はしない

テスト (**magnet_test**)

ID	内容	合格条件
T1	ランダム 100 組で dipoleForce と $-\nabla \text{dipoleEnergy}$ (中心差分) を比較。 あわせて dipoleGradient ・ m₂ == dipoleForce 、G が対称でトレース 0 であることも確認	相対誤差 < 1e-3
T2	磁石 2 個、重力と接触なしで 10 秒。運動量と角運動量 (自転 + 公転) を確認	運動量 < 1e-4、 角運動量 < 1e-3
T3	磁石 16 個、重力なし、減衰 0、反発 1、 restitutionThreshold = 0、摩擦 0 で 20 秒。substeps 1/2/4/8/16 でエネルギーのずれを記録し、CSV に出す	substeps 8 で < 1%
T4	鎖と輪 (N = 2~12) の potentialEnergy() を付録 A.1 と比較	< 1e-4
T5	一直線に接触した 2 球 (重力なし) で、 normalImpulse / h を $6\text{m}^2/\text{d}^4$ と比較	2% 以内
T6	円弧 N = 3~10 を放し、5 秒後の structures() を見る	記録のみ
I1	磁石と鉄球 (線形、飽和なし) を軸上に距離 r で置き、力を解析値 $-12\alpha\text{m}^2/r^7$ と比較。作用・反作用も確認	1e-3
I2	磁石の近くで接触した砂鉄 10 個。反復ごとの残差 $\max m - f(B(m)) $	単調に減り、20 回で < 1e-5
I3	磁石 + 鉄球で、 $U = -\frac{1}{2}\sum p \cdot B$ の中心差分 (位置ごとに m を解き直す) と力を比較	1e-3
I4	磁石 + 砂鉄 50 個、重力なしで 5 秒。誘導があっても運動量が保存するか	< 1e-4

実装の順番

- ビルド基盤: **CMakeLists.txt** と **core_test.cpp** のコピー。未変更の状態で core_test が 13/13 通ることを確認する
- magnet** の純粋関数と T1, T4
- MagnetSystem** (統一式)、**stepMagnetic**、**computeEnergy**。T2, T3, T5, T6, I1~I4
- デモのシーン 5・6、N/S 描画、HUD、磁石弾
- シーン 7 (砂鉄)。粒数を調整する
- MAGNET_SPEC.md** を実装に合わせて書き直す
 - 渦電流、M0、Physics Range、砂鉄の描画表示を削除する
 - 場の方式、統一式 (誘導磁化)、**stepMagnetic** による外側のサブステップを反映する
 - シーン 7 (砂鉄)、テストの I1~I4 を追加する
 - 「空間ハッシュは将来の拡張」と明記する

- 付録 A.3 の渦電流の参照値も削除する (tools/reference_values.py は変更しない)

今回やらないこと

外部磁場、録画機能 (--record)、空間ハッシュと高速化、渦電流、Physics Range への組み込み

検証

- ビルド (PowerShell) :& "C:\Program Files\CMake\bin\cmake.exe" -S . -B build -G "Visual Studio 17 2022" → & "C:\Program Files\CMake\bin\cmake.exe" --build build --config Release
- build\Release\core_test.exe が 13/13、build\Release\magnet_test.exe が全項目通過すること。物理コアは変更しないので、core_test は変更前後で同じ結果になるはず
- demo.exe で目視確認
 - シーン 1~4 が今までどおり動く
 - シーン 5 で鎖と輪ができる
 - シーン 6 で $N \geq 4$ が輪になるか (外れても記録)
 - シーン 7 で磁力線に沿ったトゲと極への集まりが見え、M で模様が変わる
 - HUD の physics ms が 1 フレームに収まる

リスクと対策

リスク	対策
砂鉄が磁石表面で震える、飛び込みが速すぎる	砂鉄に linearDamping を与える、シーン 7 だけ substeps 16、粒を大きくする
シーン 7 が重い、または粒が少なくて模様が見えにくい	粒の数と、シーン 7 の substeps を調整する。それでも足りなければ、次の段階で空間ハッシュ (broadphase と磁気の近傍探索の両方) を入れる
T3 で接触と位置補正がエネルギーを出し入れする	restitutionThreshold = 0 にする。ずれの内訳を README に書く
T6 で $N = 4$ が閉じない	摩擦 0 で再試行する。それでも閉じなければエネルギー障壁として考察に書く